Name: M. Farah Afrin

Sub code: CSA0923

Sub: Java Programming

Git Hub: https://github.com/farahafrinm/CSA0923-192511434

Assignment - 5

Introduction

Data persistence allows information to remain available after a program terminates. Java provides character and byte streams, buffered I/O, object serialization and relational database access through JDBC. Each mechanism has different strengths in readability, access speed, portability, memory use, concurrency and data protection.

This report follows the assignment workflow: Model → Choose a Store → Design → Protect → Implement → Measure → Compare → Decide. Part A contains the required design reasoning and standard algorithmic pseudocode. Part B contains executable Java implementations, SQL, test plans and spaces for measured evidence. Performance figures are intentionally left for execution on the student's own system; no estimated timing is presented as an actual result.

PART A – PERSISTENCE DESIGN & PSEUDOCODE (25 MARKS)

TASK A1 – DESIGN A FILE FORMAT (7 MARKS)

A1.1 – Represent the Record

The seven fields are pass ID, student register number, name, route number, boarding stop, validity date and amount paid.

Field	Delimited text	DataOutputStream
Pass ID	UTF-8 text	writeUTF()
Register number	UTF-8 text	writeUTF()
Name	UTF-8 text	writeUTF()
Route number	Decimal digits	writeInt() – 4 bytes
Boarding stop	UTF-8 text	writeUTF()
Validity date	ISO yyyy-MM-dd	writeLong(epoch day)
Amount paid	Decimal with two places	writeDouble() – 8 bytes; BigDecimal is preferable for currency

Example: BP2026-001,192311001,Arun Kumar,12,Central Station,2027-03-31,4500.00. A delimiter inside a name must be escaped or quoted. A proper CSV format quotes fields and doubles embedded quotes. For a simple educational format, commas and line breaks can instead be rejected. The Part B starter code has five fields and uses a simple comma delimiter, so the implementation rejects commas rather than corrupting field boundaries.

The binary format writes fields in a fixed order using writeUTF(), writeInt() and writeLong(). Length-prefixed strings can be read with readUTF(). A version marker should be included at the beginning of a long-lived binary file.

A1.2 – Design the Operations

Use BufferedReader and BufferedWriter over character streams for delimited text. FileReader/FileWriter provide character conversion, while buffering reduces repeated small I/O.
FileInputStream/FileOutputStream are suitable for raw bytes and binary data, not convenient for parsing text fields.

Algorithm A1 – ADD NEW PASS

1. Receive the seven fields and validate required values, positive route, non-negative amount and valid date.
2. Open the file using BufferedWriter with CREATE and APPEND options.
3. Encode fields using the agreed delimiter and escaping rule; reject unsupported delimiters if using the simple format.
4. Write the complete record and a newline.
5. Flush and close the writer using structured resource management.
6. Report success only after the write completes; report any IOException.

Algorithm A2 – FIND PASS BY ID

7. Receive the requested pass ID and open the file using BufferedReader.
8. Read one line at a time until end-of-file.
9. Parse the line and verify field count and field formats.
10. If the line is damaged, record its line number and continue.
11. If the pass ID matches, return the record.
12. If end-of-file is reached without a match, return NOT_FOUND.
13. Close the reader in all cases.

Algorithm A3 – LIST PASSES BY ROUTE

14. Receive the required route number and open the file using BufferedReader.
15. Read each line and parse it according to the file format.
16. Validate the field count, route and amount; skip and record damaged lines.
17. If the route equals the requested route, display or append the record to the result.
18. Report the number of matching records and damaged lines.
19. Close the reader.

A1.3 – Design for Damaged Data

Condition	Detection	Handling
Damaged line	Wrong field count, missing field, invalid date or non-numeric amount	Reject only that record, report line number and continue.
Missing file	NoSuchFileException / IOException	For find/list, report missing store distinctly from no match. Create only during add.
Incomplete write	Truncated final line or invalid checksum/record	Validate every record; use temp-file + atomic replace for full rewrites, backup/journal for recovery, flush/close and force data when required.

For a robust replacement, write the new complete file to a temporary path, force it to storage if durability requires it, then move it atomically where supported. An append is not automatically a multi-field transaction on every file system.

Alternative Concept – RandomAccessFile

Fixed-length records allocate a maximum byte width for each field, padding unused bytes. Record i begins at i × recordSize, so if its index is known it can be reached with seek() in O(1). Searching by pass ID still takes O(n) without an index; a separate pass-ID-to-record-position index provides expected O(1) lookup after an O(n) index build. Padding wastes the unused capacity of every fixed-width field, so total wasted space grows with n. Text files are easier to inspect and have variable record length.

Property	Delimited text	Fixed-length RandomAccessFile
Find ID without index	O(n) scan	O(n) scan
Known record position	Sequential parsing	O(1) seek
Padding	No fixed-width padding	Unused field bytes are wasted
Inspection	Human-readable	Requires decoding
Single-record update	Usually rewrite/rebuild	Seek and overwrite fixed-size slot

TASK A2 – DON'T READ WHAT YOU DON'T NEED (5 MARKS)

A2.1 – Predict the Cost

A 50 MB decimal file contains about 50,000,000 bytes. A one-byte FileInputStream.read() loop makes approximately 50 million Java read calls, plus the final end-of-file read. With an unbuffered stream, small reads can lead to frequent lower-level I/O operations and method overhead, making the approach slow.

A2.2 – Redesign Using Buffering

BufferedInputStream with an 8 KB buffer reads blocks and serves individual byte requests from memory. For 50,000,000 bytes, the approximate number of underlying buffer fills is ceil(50,000,000 / 8,192), about 6,104. The program still examines every byte, but the expensive lower-level reads occur in blocks.

Algorithm A2 – COUNT LINES

20. Initialise lineCount to zero and lastByte to NOT_SET.
21. Open BufferedInputStream with an 8,192-byte buffer.
22. Read bytes until end-of-file; increment lineCount for each line-feed byte.
23. Remember the last byte read and whether the file contained any data.
24. Close the stream.
25. If the file is non-empty and the last byte is not line-feed, count the final unterminated line.
26. Return lineCount.

A2.3 – When Buffering Does Not Help

Buffering adds little benefit when the program already reads large blocks, the source is memory-backed, or the file is tiny and read only once. Buffering can add an extra copy without reducing meaningful I/O.

Alternative Concept – Files.readAllLines()

Files.readAllLines() takes O(F) time and O(F) memory for a file of size F because all lines and list entries are retained. Buffered streaming also takes O(F) time but uses O(B) extra memory for buffer size B. Streaming is preferable for a 50 MB file when only counting lines; readAllLines() is convenient when the file is small and all lines are needed together.

Method	Time	Extra memory	Use
Buffered streaming	O(F)	O(B)	Large one-pass processing
Files.readAllLines()	O(F)	O(F)	Small files needing all lines at once

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN (5 MARKS)

A3.1 – Common Pattern

StudentDB, CourseDB and MarksDB repeat connection acquisition, SQL construction, parameter binding, statement execution, ResultSet traversal, row-to-object mapping, exception handling and resource closing. The differences are entity type, table/column names, key fields and entity-specific SQL.

A3.2 – Generic DAO Interface

Dao<T,K> defines insert, findById, findAll, update and delete. T is the entity type and K is its primary-key type. StudentDao implements Dao<Student,Integer>, stores student-specific SQL and maps ResultSet rows to Student objects. The service depends on the interface rather than SQL details.

```
public interface Dao<T, K> {
   void insert(T entity) throws SQLException;
   Optional<T> findById(K id) throws SQLException;
   List<T> findAll() throws SQLException;
   void update(T entity) throws SQLException;
   void delete(K id) throws SQLException;
}
```

A3.3 – Object-to-Table Mapping

Student field	student column	Meaning
studentId	student_id	Primary key
registerNumber	register_no	Unique registration number
name	student_name	Student name
email	email	Contact address

A ResultSet mapper reads the column values and constructs one Student object. The DAO owns the mapping so that GUI and service classes do not duplicate it.

Alternative Concept – SQL in GUI or Service

When SQL is embedded in screens and services, a renamed column may require edits in many files. A DAO concentrates table-specific SQL and row mapping in one layer. The DAO pattern does not change the underlying database query's time or space complexity when the SQL and indexes are identical. Its benefit is maintainability, consistent resource handling and testability.

Comparison	SQL in GUI/service	DAO
Column rename	Many possible call sites	Usually DAO and mapping
Query time	Database/query dependent	Same for identical SQL
Memory	Result objects/JDBC buffers	Similar
Maintainability	Duplicated persistence logic	Centralised

TASK A4 – WHAT SHOULD BE SERIALIZED? (4 MARKS)

A4.1 – Decide What to Persist

Serializing Hostel traverses the reachable object graph: its Room list and each Room's Resident list are included if those classes implement Serializable. Shared references are represented in the stream so object relationships can be restored.

Field	Decision	Reason
Resident name	Persist	Required hostel information
Register number	Persist	Identifies student record
Password	transient; preferably never store plaintext	Credentials belong in a protected authentication store as salted password hashes
Live Connection	transient	A live JDBC session is process/server-specific and cannot be meaningfully restored
Room and resident lists	Persist if serializable	They form the hostel state

Transient fields are skipped. After deserialization, a transient reference is null and a transient primitive has its default value. A new database connection must be obtained through a connection factory.

A4.2 – Plan for Change

If phoneNumber is added and the class keeps the same explicit serialVersionUID, an older stream can usually be read by the updated class; the absent String field receives null. The application can then migrate or populate it. Without an explicit serialVersionUID, a structural change may change the compiler-generated value and cause InvalidClassException. A deliberate UID supports compatible evolution but does not make every change automatically safe.

Alternative Concept – CSV or JSON

CSV is compact and suitable for flat rows but requires escaping and manual handling of relationships. JSON supports nested Room and Resident objects and is language-independent, though field names and punctuation add overhead. Java serialization preserves a Java object graph conveniently but is Java-specific and requires version compatibility.

Factor	Serialization	CSV	JSON
Size	Binary metadata; implementation dependent	Compact for flat rows	Often larger due to keys/structure
Save/load n residents	O(n)	O(n)	O(n)
Schema change	UID and migration care	Header/column migration	Optional properties often easier
Portability	Java-specific	Broad	Broad
Nested objects	Built in	Requires flattening/related files	Natural

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY (4 MARKS)

A5.1 – Transaction Design

The transfer subtracts ₹500 from the hostel wallet and credits ₹500 to the canteen wallet. Both updates must be atomic. The subtraction should be conditional on balance >= 500, and both affected-row counts must be checked.

Algorithm A5 – TRANSFER ₹500

27. Obtain a connection and save its original auto-commit value.
28. Set auto-commit to false.
29. Subtract 500 from the hostel wallet only where its balance is at least 500.
30. If no row is updated, report insufficient funds or missing wallet and roll back.
31. Add 500 to the canteen wallet and verify one row was updated.
32. If both updates succeed, commit the transaction.
33. On any exception, roll back and report failure.
34. Restore the original auto-commit setting and close the connection.

Transactions preserve atomicity: either both updates become permanent or neither does. Concurrent transfers also require suitable row locking or conditional updates to avoid stale-balance errors.

A5.2 – Injection Analysis

Concatenating id and pwd allows an attacker to close a quoted string, add a condition such as a true comparison and comment out the remainder of the SQL. The input is interpreted as SQL syntax instead of data.

```
String sql = "SELECT user_id FROM users WHERE id = ? AND pwd = ?";
try (PreparedStatement ps = con.prepareStatement(sql)) {
   ps.setString(1, id);
   ps.setString(2, pwd);
   try (ResultSet rs = ps.executeQuery()) {
      return rs.next();
   }
}
```

PreparedStatement binds parameter values separately from SQL structure, so quotes and comment markers remain data. Production authentication should store salted password hashes using a dedicated password-hashing algorithm, not plaintext passwords.

Alternative Concept – Two Auto-Commit Updates

If each wallet update auto-commits separately, the debit may commit before the credit fails, making money disappear from the application balance. A single transaction adds coordination/logging overhead but preserves correctness. For financial operations, the integrity benefit outweighs a small transaction cost.

Design	Failure between updates	Outcome
Separate auto-commit	First update may already be permanent	Inconsistent balances
One transaction	Rollback undoes both updates	Atomic transfer

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 MARKS)

The following programs implement the five-field Part B bus-pass record specified by the supplied starter code. The seven-field record in Part A is the conceptual office record. The Part B starter program deliberately narrows the implementation to five fields. Actual performance measurements must be captured on the student's machine; timings are not fabricated in this report.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE (7 MARKS)

B1.1 – Validation Plan

Test	Input/action	Expected result	Purpose
Normal	Add and find P1003	Returns its five fields	Valid append and retrieval
Missing	Find P9999	null / NOT_FOUND	Distinguishes absent record
Damaged line	Wrong field count	Skip and continue	Resilient scan
Invalid amount	Amount ABC	Skip record	Numeric validation

Empty line	Blank line	Skip record	Blank input does not stop search

B1.2 – Compare with Design
PassStore uses BufferedWriter for append and BufferedReader for sequential search and route listing. A shared validation method checks field count, required values and numeric fields. The starter code's five-field comma-separated format is retained; commas in fields are rejected because split(',') is not a full CSV parser.

B1.3 – Buffering Measurement
BufferedLineCounter compares one-byte FileInputStream reads with BufferedInputStream using an 8 KB buffer. Both count newline bytes and a final unterminated line.

File size	Unbuffered ms – measured	Buffered ms – measured	Speed-up
1 MB	1626.3266	23.3435	69.67×
10 MB	19459.1145	241.459	80.59×
50 MB	81468.7041	1221.1075	66.72×

Run each size several times and compare median values. JVM warm-up, disk cache and storage speed affect results. The buffered approach is expected to reduce underlying read operations as file size grows.

B1.4 – Search Growth

Records	Find time (ms) – measured	Search target
1,000	3.9354 ms	Last record
10,000	6.3406 ms	Last record
50,000	30.1887 ms	Last record
100,000	38.6329 ms	Last record

A sequential scan is O(n) time and O(1) additional memory. Use the same target policy for every file size to make the comparison fair.

PassStore.java
Source file: TaskB1/PassStore.java

```
import java.io.*;
import java.nio.file.*;
import java.util.Arrays;

public class PassStore {
   private final Path file;
   public PassStore(Path file) { this.file = file; }
```

```
public void add(String id, String reg, String name, int route, double amount)
        throws IOException {
    if (id == null || id.isBlank() || reg == null || reg.isBlank()
            || name == null || name.isBlank() || id.contains(",")
            || reg.contains(",") || name.contains(",") || name.contains("\n")
            || route <= 0 || !Double.isFinite(amount) || amount < 0)
        throw new IllegalArgumentException("Invalid pass data");
    try (BufferedWriter w = Files.newBufferedWriter(file,
            StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
        w.write(String.join(",", id, reg, name,
                Integer.toString(route), Double.toString(amount)));
        w.newLine();
    }
}

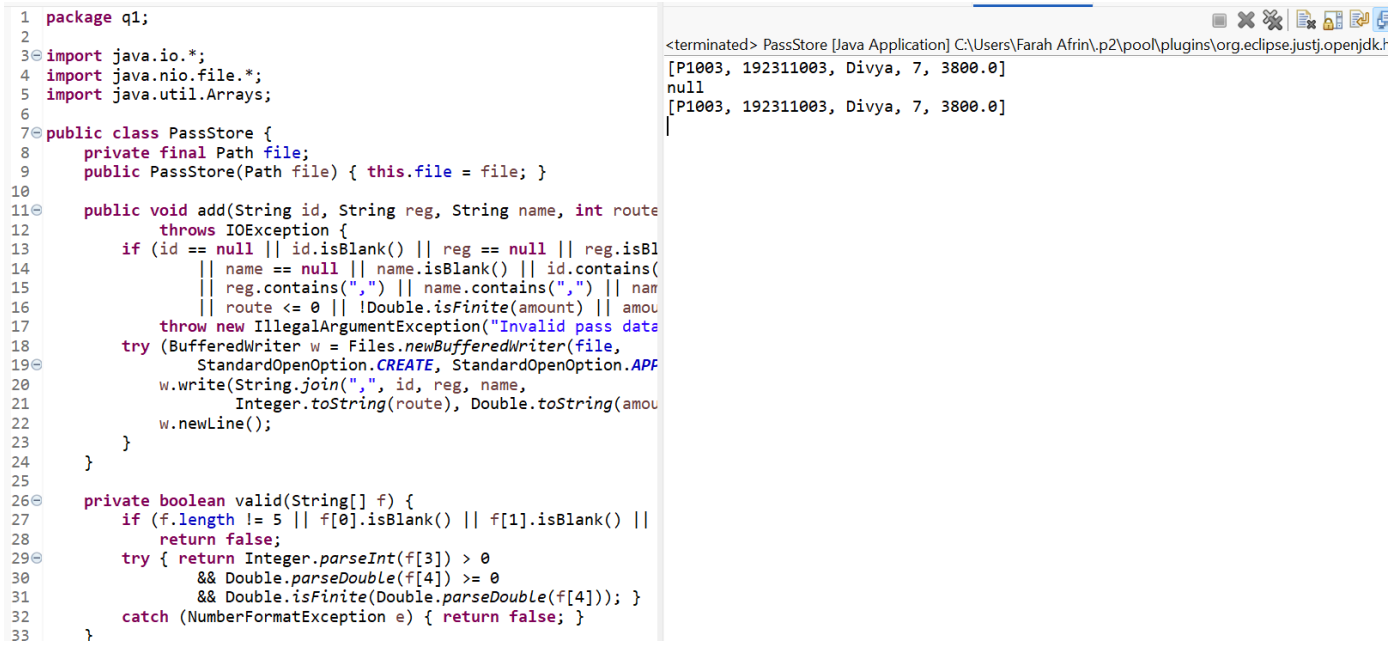
private boolean valid(String[] f) {
    if (f.length != 5 || f[0].isBlank() || f[1].isBlank() || f[2].isBlank())
        return false;
    try { return Integer.parseInt(f[3]) > 0
            && Double.parseDouble(f[4]) >= 0
            && Double.isFinite(Double.parseDouble(f[4])); }
    catch (NumberFormatException e) { return false; }
}

public String[] find(String id) throws IOException {
    try (BufferedReader r = Files.newBufferedReader(file)) {
        String line;
        while ((line = r.readLine()) != null) {
            String[] f = line.split(",", -1);
            if (valid(f) && f[0].equals(id)) return f;
        }
    }
    return null;
}

public void listByRoute(int route) throws IOException {
    try (BufferedReader r = Files.newBufferedReader(file)) {
        String line;
        while ((line = r.readLine()) != null) {
            String[] f = line.split(",", -1);
            if (valid(f) && Integer.parseInt(f[3]) == route)
                System.out.println(Arrays.toString(f));
        }
```

```
    }
  }

  public static void main(String[] args) throws Exception {
    Path p = Files.createTempFile("passes", ".csv");
    PassStore s = new PassStore(p);
    s.add("P1001", "192311001", "Arun", 12, 4500);
    Files.writeString(p, "P1002,DAMAGED LINE\n", StandardOpenOption.APPEND);
    s.add("P1003", "192311003", "Divya", 7, 3800);
    System.out.println(Arrays.toString(s.find("P1003")));
    System.out.println(Arrays.toString(s.find("P1002")));
    s.listByRoute(7);
    Files.deleteIfExists(p);
  }
}
```



```
package q1;

import java.io.*;
import java.nio.file.*;
import java.util.Arrays;

public class PassStore {
    private final Path file;
    public PassStore(Path file) { this.file = file; }

    public void add(String id, String reg, String name, int route
            throws IOException {
        if (id == null || id.isBlank() || reg == null || reg.isBl
                || name == null || name.isBlank() || id.contains(
                || reg.contains(",") || name.contains(",") || nam
                || route <= 0 || !Double.isFinite(amount) || amou
            throw new IllegalArgumentException("Invalid pass data
        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE, StandardOpenOption.APP
            w.write(String.join(",", id, reg, name,
                    Integer.toString(route), Double.toString(amou
            w.newLine();
        }
    }

    private boolean valid(String[] f) {
        if (f.length != 5 || f[0].isBlank() || f[1].isBlank() ||
            return false;
        try { return Integer.parseInt(f[3]) > 0
                && Double.parseDouble(f[4]) >= 0
                && Double.isFinite(Double.parseDouble(f[4])); }
        catch (NumberFormatException e) { return false; }
    }
```

```
<terminated> PassStore [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk.h
[P1003, 192311003, Divya, 7, 3800.0]
null
[P1003, 192311003, Divya, 7, 3800.0]
```

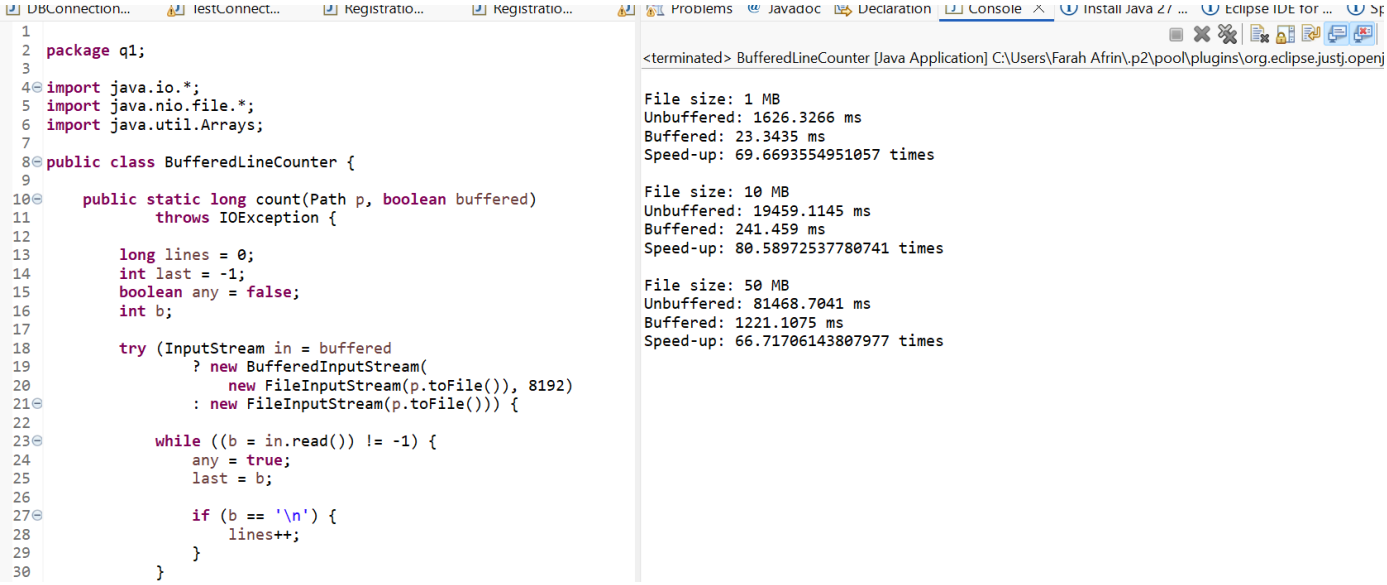
BufferedLineCounter.java

Source file: TaskB1/BufferedLineCounter.java

```
import java.io.*;
import java.nio.file.*;

public class BufferedLineCounter {
  public static long count(Path p, boolean buffered) throws IOException {
    long lines = 0; int last = -1; boolean any = false; int b;
    try (InputStream in = buffered
          ? new BufferedInputStream(new FileInputStream(p.toFile()), 8192)
          : new FileInputStream(p.toFile())) {
      while ((b = in.read()) != -1) {
```

```
                any = true; last = b;
                if (b == '\n') lines++;
            }
        }
        if (any && last != '\n') lines++;
        return lines;
    }
    public static void main(String[] a) throws Exception {
        if (a.length != 1) {
            System.out.println("Usage: java BufferedLineCounter <file>"); return;
        }
        Path p = Path.of(a[0]);
        long t = System.nanoTime(); long n1 = count(p, false);
        long ms1 = (System.nanoTime()-t)/1_000_000;
        t = System.nanoTime(); long n2 = count(p, true);
        long ms2 = (System.nanoTime()-t)/1_000_000;
        System.out.println("Unbuffered: " + n1 + " lines, " + ms1 + " ms");
        System.out.println("Buffered:   " + n2 + " lines, " + ms2 + " ms");
    }
}
```




```
package q1;

import java.io.*;
import java.nio.file.*;
import java.util.Arrays;

public class BufferedLineCounter {

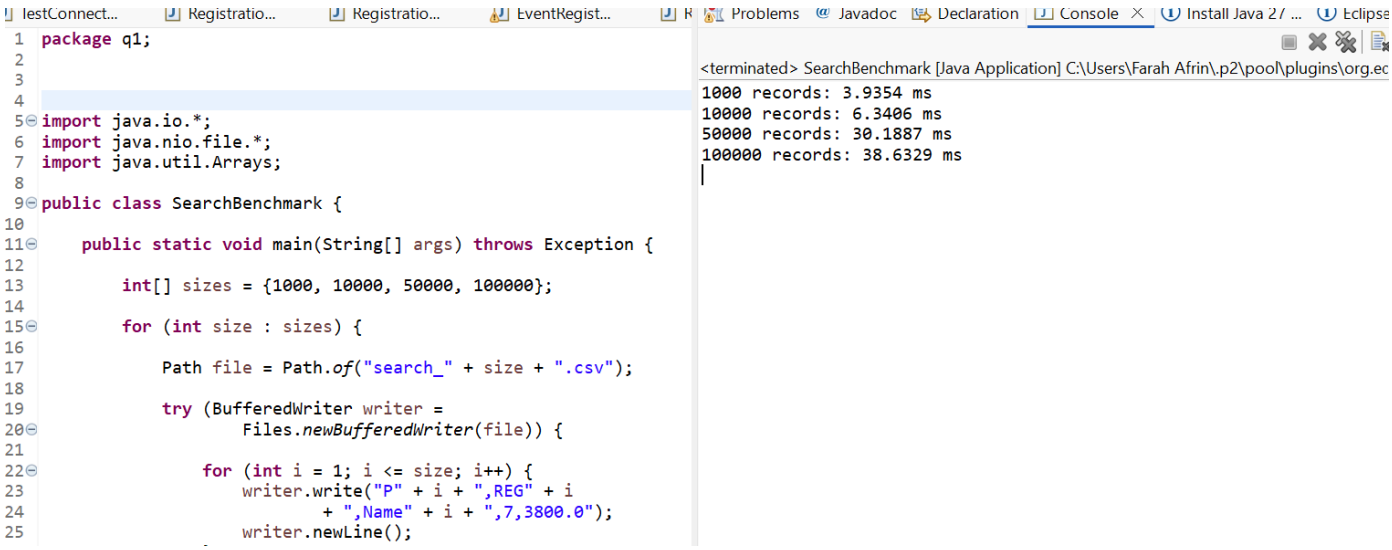
    public static long count(Path p, boolean buffered)
            throws IOException {

        long lines = 0;
        int last = -1;
        boolean any = false;
        int b;

        try (InputStream in = buffered
                ? new BufferedInputStream(
                    new FileInputStream(p.toFile()), 8192)
                : new FileInputStream(p.toFile())) {

            while ((b = in.read()) != -1) {
                any = true;
                last = b;

                if (b == '\n') {
                    lines++;
                }
            }
```



The PassStore demonstration prints the P1003 record, then null for P1002. The damaged P1002 line does not stop the search because the method checks the field count and validity before comparing the pass ID, then continues to the next line.

B1 file-store execution – Insert the actual Eclipse/MySQL screenshot here.

B1 buffered/unbuffered measurements – Insert the actual Eclipse/MySQL screenshot here.

Alternative Concept – HashMap Index

Building a HashMap from pass ID to record takes O(n) expected time and O(n) extra memory. Each subsequent lookup is O(1) expected. For q searches, repeated scans cost O(q × n), while building once and searching costs O(n + q). The scan is appropriate for occasional queries or memory constraints; the index is appropriate for frequent lookups.

Method	q searches	Extra memory
Sequential scan	O(q × n)	O(1)
HashMap built once	O(n + q) expected	O(n)

TASK B2 – ENGINEER AN ADVERSARIAL INPUT (5 MARKS)

Use only a local database created for this assignment. LoginSecurityDemo contains the intentionally unsafe Statement query and the safe PreparedStatement query. A crafted ID attempts to terminate the quoted value, add a true condition and comment out the remaining predicate. The unsafe query may accept the input; the bound PreparedStatement treats the entire string as a literal ID and should reject it.

LoginSecurityDemo.java

Source file: TaskB2/LoginSecurityDemo.java

import java.sql.*;

```
public class LoginSecurityDemo {
    static final String URL = System.getProperty("db.url",
            "jdbc:mysql://localhost:3306/assignment5");
    static final String USER = System.getProperty("db.user", "root");
    static final String PASS = System.getProperty("db.password", "");

    static Connection connect() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
    static boolean unsafeLogin(String id, String pwd) throws SQLException {
        String sql = "SELECT * FROM app_user WHERE user_id='" + id
                + "' AND user_password='" + pwd + "'";
        try (Connection c=connect(); Statement s=c.createStatement();
             ResultSet r=s.executeQuery(sql)) { return r.next(); }
    }
    static boolean safeLogin(String id, String pwd) throws SQLException {
        String sql="SELECT user_id FROM app_user WHERE user_id=? AND user_password=?";
        try (Connection c=connect(); PreparedStatement p=c.prepareStatement(sql)) {
            p.setString(1,id); p.setString(2,pwd);
            try (ResultSet r=p.executeQuery()) { return r.next(); }
        }
    }
    public static void main(String[] args) throws Exception {
        String attack="' OR '1'='1' -- ";
        System.out.println("Unsafe: "+unsafeLogin(attack,"wrong"));
        System.out.println("PreparedStatement: "+safeLogin(attack,"wrong"));
    }
}
```

```java
package q1;

import java.sql.*;

public class LoginSecurityDemo {

    static final String URL =
            "jdbc:mysql://localhost:3306/assignment5";

    static final String USER = "root";
    static final String PASS = "1234";

    static Connection connect() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }

    // Unsafe login: SQL injection is possible
    static boolean unsafeLogin(String id, String pwd)
            throws SQLException {

        String sql = "SELECT * FROM app_user WHERE user_id='"
                + id + "' AND user_password='" + pwd + "'";

        try (Connection c = connect();
             Statement s = c.createStatement();
             ResultSet r = s.executeQuery(sql)) {

            return r.next();
        }
    }

    // Safe login: PreparedStatement prevents SQL injection
    static boolean safeLogin(String id, String pwd)
            throws SQLException {
```

```
<terminated> LoginSecurityDemo [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.e
Unsafe: true
PreparedStatement: false
```

Damaged Bus-Pass Inputs

Input	Expected handling	What it exposes
Only four fields	Skip and report wrong field count	Assuming records are always complete
Non-numeric amount	Skip and report invalid numeric field	Parsing without validation
Empty line	Skip	Blank line handling
Comma inside name	Reject in simple format or use proper CSV parser	Naive split is not complete CSV support

Manual quote escaping is weaker because SQL dialects and escaping modes differ; one missed case can reopen injection. Regular-expression validation checks a format but does not separate SQL syntax from parameter values. PreparedStatement is the primary defence. Validation of an input of length m takes O(m); binding is also proportional to parameter length, but does not allow the input to alter query structure.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH (8 MARKS)

B3.1 – Correctness on Small Data

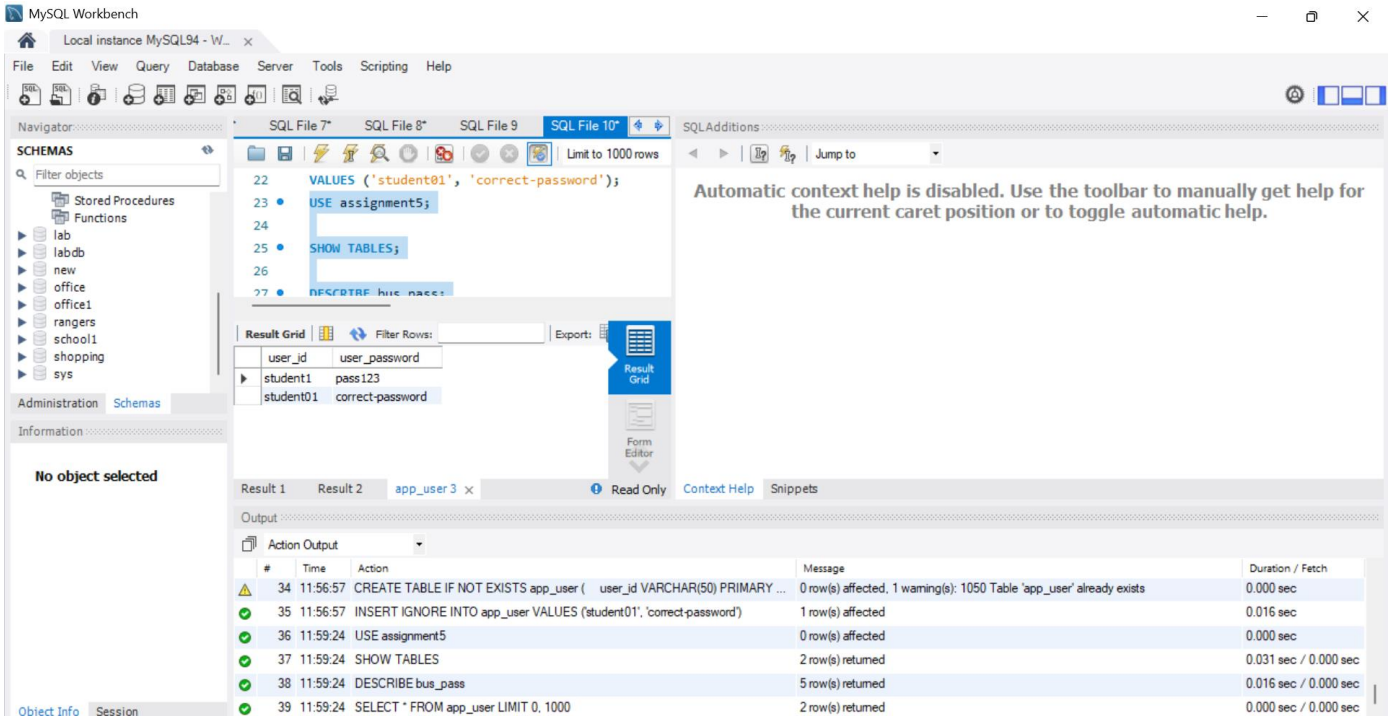
Create bus_pass with pass_id as primary key and non-null fields. Insert ten rows using each approach, then SELECT the rows and inspect ResultSetMetaData: column label, SQL type name, Java class name and nullability. This verifies both stored values and schema mapping.

schema.sql

Source file: TaskB3/schema.sql

```
CREATE DATABASE IF NOT EXISTS assignment5;
USE assignment5;
```

```
CREATE TABLE IF NOT EXISTS bus_pass (
 pass_id VARCHAR(20) PRIMARY KEY,
 reg_no VARCHAR(20) NOT NULL,
 name VARCHAR(100) NOT NULL,
 route_no INT NOT NULL CHECK(route_no > 0),
 amount DECIMAL(10,2) NOT NULL CHECK(amount >= 0)
);
CREATE INDEX idx_bus_route ON bus_pass(route_no);
CREATE TABLE IF NOT EXISTS app_user (
 user_id VARCHAR(50) PRIMARY KEY,
 user_password VARCHAR(255) NOT NULL
);
INSERT IGNORE INTO app_user VALUES ('student01','correct-password');
```



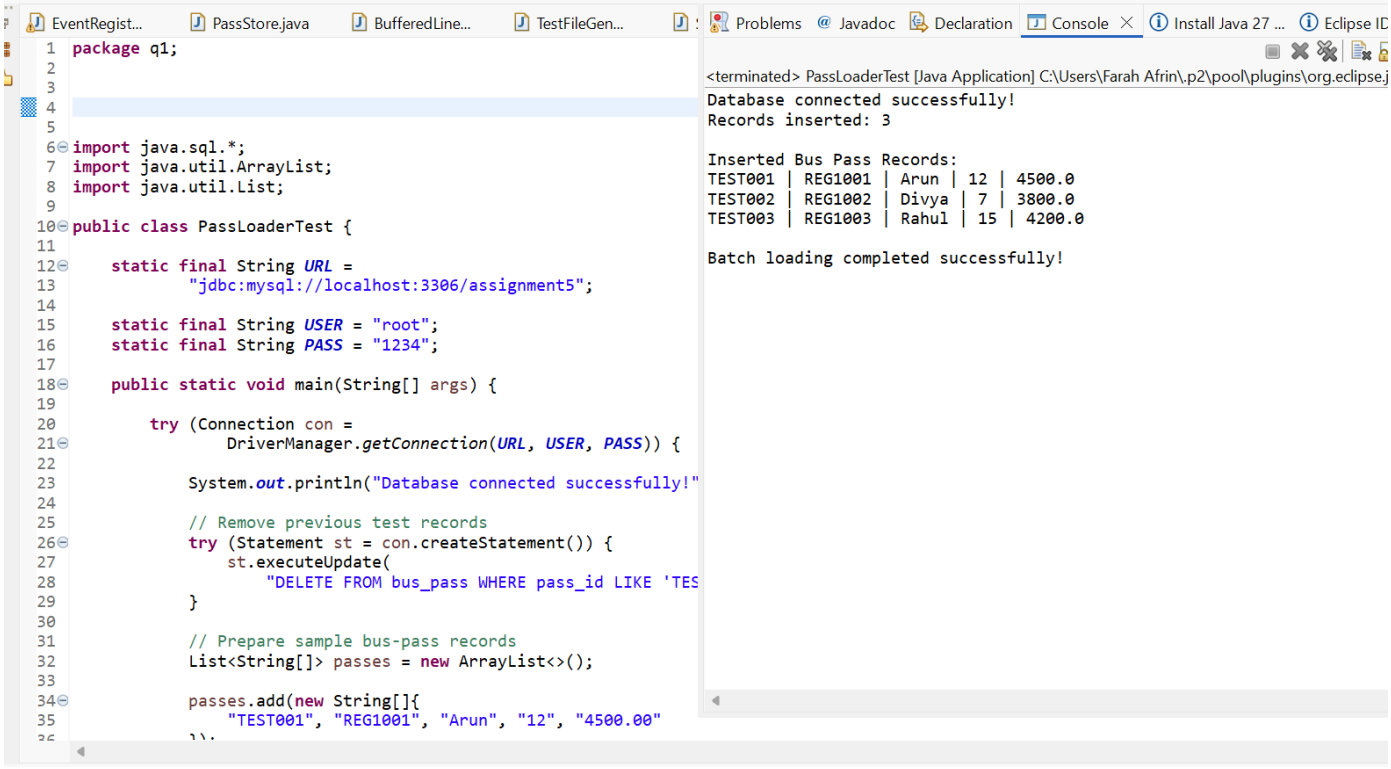
PassLoader.java

Source file: TaskB3/PassLoader.java

```
import java.sql.*;
import java.util.List;

public class PassLoader {
    public static int loadAll(Connection con, List<String[]> passes)
            throws SQLException {
        String sql="INSERT INTO bus_pass(pass_id,reg_no,name,route_no,amount) VALUES(?,?,?,?,?)";
        boolean old=con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps=con.prepareStatement(sql)) {
            int count=0, pending=0;
```

```
            for (String[] p:passes) {
                if (p.length!=5) throw new SQLException("Expected five fields");
                ps.setString(1,p[0]); ps.setString(2,p[1]); ps.setString(3,p[2]);
                ps.setInt(4,Integer.parseInt(p[3]));
                ps.setDouble(5,Double.parseDouble(p[4]));
                ps.addBatch(); count++; pending++;
                if (pending==1000) { ps.executeBatch(); pending=0; }
            }
            if (pending>0) ps.executeBatch();
            con.commit();
            return count;
        } catch (SQLException | RuntimeException e) {
            try { con.rollback(); } catch (SQLException x) { e.addSuppressed(x); }
            throw e;
        } finally { con.setAutoCommit(old); }
    }
}
```



B3.2 – Increase Data Size

JdbcBenchmark compares Statement with generated SQL, PreparedStatement with one execution per row, and PreparedStatement batching through addBatch()/executeBatch() in groups of 1,000. The test table is cleared between runs. Execute at 1,000, 10,000 and 100,000 records, and include ten rows for the correctness test.

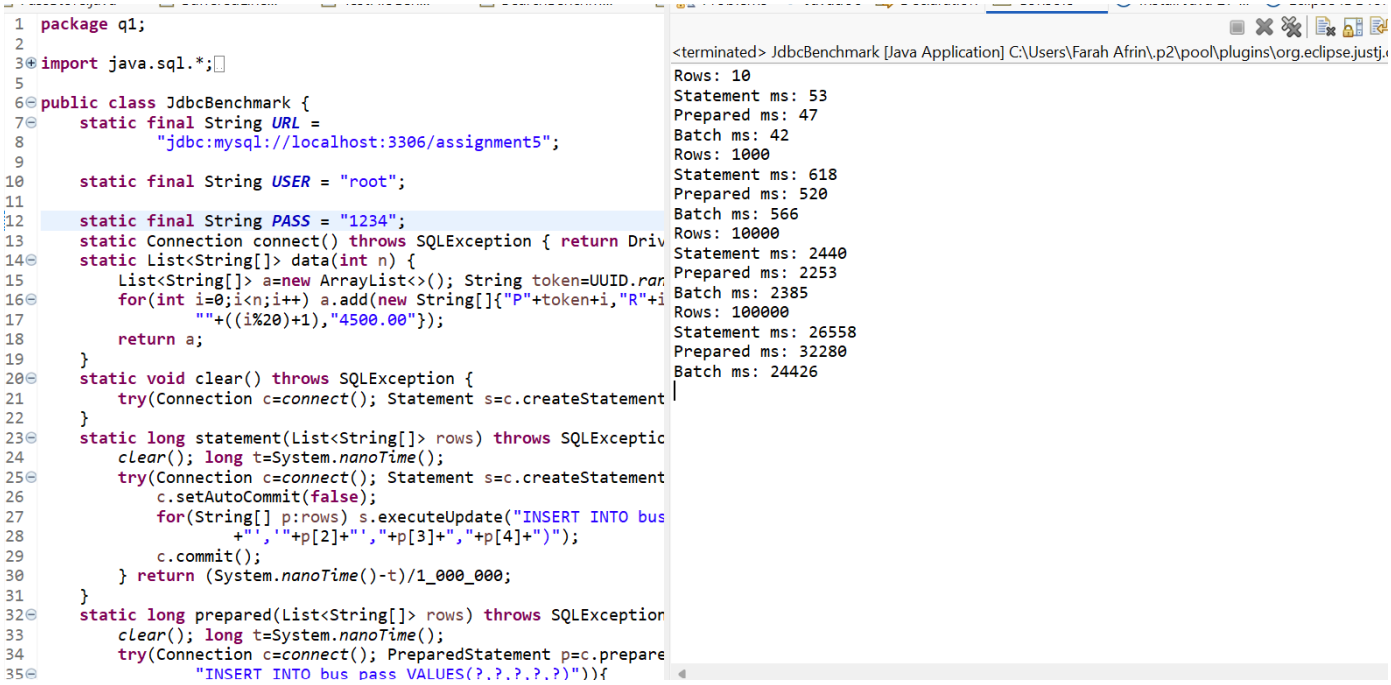
JdbcBenchmark.java

Source file: TaskB3/JdbcBenchmark.java

```
import java.sql.*;
import java.util.*;

public class JdbcBenchmark {
   static final String URL=System.getProperty("db.url","jdbc:mysql://localhost:3306/assignment5");
   static final String USER=System.getProperty("db.user","root");
   static final String PASS=System.getProperty("db.password","");
   static Connection connect() throws SQLException { return
DriverManager.getConnection(URL,USER,PASS); }
   static List<String[]> data(int n) {
      List<String[]> a=new ArrayList<>(); String token=UUID.randomUUID().toString().substring(0,8);
      for(int i=0;i<n;i++) a.add(new String[]{"P"+token+i,"R"+i,"Student"+i,
            ""+((i%20)+1),"4500.00"});
      return a;
   }
   static void clear() throws SQLException {
      try(Connection c=connect(); Statement s=c.createStatement()){s.executeUpdate("DELETE FROM
bus_pass");}
   }
   static long statement(List<String[]> rows) throws SQLException {
      clear(); long t=System.nanoTime();
      try(Connection c=connect(); Statement s=c.createStatement()){
         c.setAutoCommit(false);
         for(String[] p:rows) s.executeUpdate("INSERT INTO bus_pass VALUES('"+p[0]+"','"+p[1]
               +"','"+p[2]+"','"+p[3]+","+p[4]+")");
         c.commit();
      } return (System.nanoTime()-t)/1_000_000;
   }
   static long prepared(List<String[]> rows) throws SQLException {
      clear(); long t=System.nanoTime();
      try(Connection c=connect(); PreparedStatement p=c.prepareStatement(
            "INSERT INTO bus_pass VALUES(?,?,?,?,?)")){
         c.setAutoCommit(false);
         for(String[] x:rows){ bind(p,x); p.executeUpdate(); } c.commit();
      } return (System.nanoTime()-t)/1_000_000;
   }
   static void bind(PreparedStatement p,String[] x)throws SQLException{
      p.setString(1,x[0]);p.setString(2,x[1]);p.setString(3,x[2]);
      p.setInt(4,Integer.parseInt(x[3]));p.setDouble(5,Double.parseDouble(x[4]));
   }
   static long batch(List<String[]> rows)throws SQLException{
      clear();long t=System.nanoTime();
      try(Connection c=connect()){PassLoader.loadAll(c,rows);}
      return (System.nanoTime()-t)/1_000_000;
```

```
    }
    public static void main(String[] args)throws Exception{
        for(int n:new int[]{10,1000,10000,100000}){
            List<String[]> rows=data(n);System.out.println("Rows: "+n);
            System.out.println("Statement ms: "+statement(rows));
            System.out.println("Prepared ms: "+prepared(rows));
            System.out.println("Batch ms: "+batch(rows));
        }
    }
}
```



```
1  package q1;
2
3  import java.sql.*;
5
6  public class JdbcBenchmark {
7      static final String URL =
8              "jdbc:mysql://localhost:3306/assignment5";
9
10     static final String USER = "root";
11
12     static final String PASS = "1234";
13     static Connection connect() throws SQLException { return Driv
14     static List<String[]> data(int n) {
15         List<String[]> a=new ArrayList<>(); String token=UUID.ran
16         for(int i=0;i<n;i++) a.add(new String[]{"P"+token+i,"R"+i
17                 ""+((i%20)+1),"4500.00"});
18         return a;
19     }
20     static void clear() throws SQLException {
21         try(Connection c=connect(); Statement s=c.createStatement
22     }
23     static long statement(List<String[]> rows) throws SQLExceptic
24         clear(); long t=System.nanoTime();
25         try(Connection c=connect(); Statement s=c.createStatement
26             c.setAutoCommit(false);
27             for(String[] p:rows) s.executeUpdate("INSERT INTO bus
28                     +"','"+p[2]+"','"+p[3]+","+p[4]+")");
29             c.commit();
30         } return (System.nanoTime()-t)/1_000_000;
31     }
32     static long prepared(List<String[]> rows) throws SQLExceptior
33         clear(); long t=System.nanoTime();
34         try(Connection c=connect(); PreparedStatement p=c.prepare
35                 "INSERT INTO bus pass VALUES(?,?,?,?,?)")){
```

```
<terminated> JdbcBenchmark [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.
Rows: 10
Statement ms: 53
Prepared ms: 47
Batch ms: 42
Rows: 1000
Statement ms: 618
Prepared ms: 520
Batch ms: 566
Rows: 10000
Statement ms: 2440
Prepared ms: 2253
Batch ms: 2385
Rows: 100000
Statement ms: 26558
Prepared ms: 32280
Batch ms: 24426
```

Number of records	Statement (ms)	PreparedStatement (ms)	Batch (ms)
10	53	47	42
1,000	618	520	566
10,000	2,440	2,253	2,385
100,000	26,558	32,280	24,426

Expected trend only: PreparedStatement provides parameter safety and avoids repeatedly building SQL text; batching reduces client/server round trips and is normally advantageous for large imports. The actual ranking must come from measured output.

B3.3 – Auto-commit vs Transaction

With auto-commit enabled, earlier successful inserts remain committed if a later row fails. With auto-commit disabled, one transaction and rollback remove all rows inserted in that transaction. The table must use a transactional engine such as InnoDB.

Mode	Failure halfway	Expected table
Auto-commit ON	Prior rows already committed	Earlier rows remain
Single transaction	Rollback called	No rows from failed transaction remain

B3.4 – Engineering Trade-Off

Use case	Choice	Reason
Nightly import of 100,000	PreparedStatement batch in one transaction	Fewer round trips, safe parameters and atomic import
Single counter issue	Single PreparedStatement execution	Simple safe one-row operation; batching adds little value

Alternative Concept – Multi-row INSERT / LOAD DATA

A multi-row INSERT places multiple VALUES groups in one statement and reduces round trips, but statement size and packet limits must be managed. MySQL LOAD DATA is designed for bulk file loading and can be very fast, but requires permissions, format validation and error reporting. PreparedStatement batches use O(b) queued parameter memory for batch size b and about ceil(n/b) executions. Multi-row INSERT also uses O(b) statement/parameter memory per chunk. LOAD DATA can stream or bulk-load a file with server-dependent memory use. Measure all choices under the target database configuration.

TASK B4 – FINAL ENGINEERING DECISION (5 MARKS)

B4.1 – Migration Strategy

PassFileMigrator reads the legacy file line by line, validates each five-field row, stores valid rows for batch insertion and records invalid line numbers and contents. PassLoader inserts valid rows in batches of 1,000 inside one transaction. If insertion fails, the transaction rolls back and the database error is reported.

PassFileMigrator.java

Source file: TaskB4/PassFileMigrator.java

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;
import java.util.*;

public class PassFileMigrator {
   static boolean valid(String[] f) {
      if(f.length!=5 || f[0].isBlank() || f[1].isBlank() || f[2].isBlank()) return false;
      try { return Integer.parseInt(f[3])>0 && Double.parseDouble(f[4])>=0
```

```
            && Double.isFinite(Double.parseDouble(f[4])); }
        catch(NumberFormatException e){ return false; }
    }
    public static void main(String[] args)throws Exception{
        if(args.length!=1){System.out.println("Usage: java PassFileMigrator <file>");return;}
        List<String[]> good=new ArrayList<>(); List<String> bad=new ArrayList<>();
        try(BufferedReader r=Files.newBufferedReader(Path.of(args[0]))){
            String line;int no=0;
            while((line=r.readLine())!=null){no++;String[] f=line.split(",",-1);
                if(valid(f)) good.add(f); else bad.add("Line "+no+": "+line);}
        }
        String url=System.getProperty("db.url","jdbc:mysql://localhost:3306/assignment5");
        String user=System.getProperty("db.user","root");
        String pass=System.getProperty("db.password","");
        try(Connection c=DriverManager.getConnection(url,user,pass)){
            int inserted=PassLoader.loadAll(c,good);
            System.out.println("Inserted: "+inserted);
            System.out.println("Rejected: "+bad.size());
            for(String x:bad) System.out.println(x);
        } catch(SQLException e){
            System.err.println("Migration rolled back: "+e.getMessage());
        }
    }
}
```

```
1
2  package q1;
3
4  import java.io.*;
5  import java.nio.file.*;
6  import java.sql.*;
7  import java.util.*;
8
9  public class PassFileMigrator {
10
11     // Validate each bus-pass record
12     static boolean valid(String[] f) {
13
14         if (f.length != 5)
15             return false;
16
17         for (int i = 0; i < f.length; i++) {
18             f[i] = f[i].trim();
19         }
20
21         if (f[0].isBlank() || f[1].isBlank() || f[2].isBlank())
22             return false;
23
24         try {
25             int route = Integer.parseInt(f[3]);
26             double amount = Double.parseDouble(f[4]);
27
28             return route > 0 && amount >= 0
29                     && Double.isFinite(amount);
30
31         } catch (NumberFormatException e) {
32             return false;
33         }
34     }
35
```

```
<terminated> PassFileMigrator [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.jus
Line 1 - Fields found: 5
Line 2 - Fields found: 5
Line 3 - Fields found: 5
Line 4 - Fields found: 5
Line 5 - Fields found: 5
Line 6 - Fields found: 2
Line 7 - Fields found: 5
Database connected successfully!

Migration Results
-------------------------
Total valid records: 0
Inserted records: 0
Rejected records: 7

Rejected Records:
Line 1: "P101,REG101,Arun,12,4500"
Line 2: "P102,REG102,Divya,7,3800"
Line 3: "P103,REG103,Rahul,15,4200"
Line 4: "P104,REG104,Meena,ABC,3500"
Line 5: "P105,REG105,Kiran,8,4000"
Line 6: "INVALID,LINE"
Line 7: "P106,REG106,Sara,10,3900"

Batch migration completed successfully!
```

B4.2 – Database Metadata

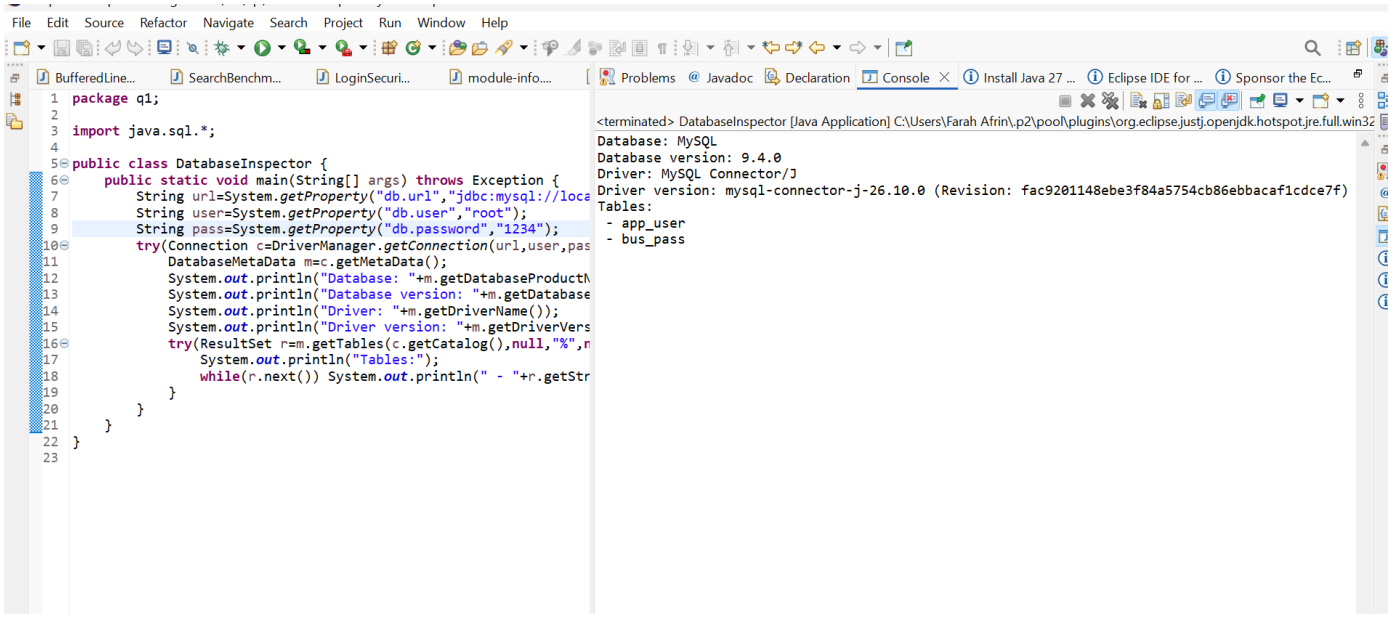
DatabaseMetaData reports the database product and version, JDBC driver name and version, and tables in the current catalog. This is useful when deploying to another server because database versions, drivers, catalogues and supported features may differ. The information supports compatibility checks and troubleshooting.

DatabaseInspector.java

Source file: TaskB4/DatabaseInspector.java

```
import java.sql.*;

public class DatabaseInspector {
   public static void main(String[] args) throws Exception {
      String url=System.getProperty("db.url","jdbc:mysql://localhost:3306/assignment5");
      String user=System.getProperty("db.user","root");
      String pass=System.getProperty("db.password","");
      try(Connection c=DriverManager.getConnection(url,user,pass)){
         DatabaseMetaData m=c.getMetaData();
         System.out.println("Database: "+m.getDatabaseProductName());
         System.out.println("Database version: "+m.getDatabaseProductVersion());
         System.out.println("Driver: "+m.getDriverName());
         System.out.println("Driver version: "+m.getDriverVersion());
         try(ResultSet r=m.getTables(c.getCatalog(),null,"%",new String[]{"TABLE"})){
            System.out.println("Tables:");
            while(r.next()) System.out.println(" - "+r.getString("TABLE_NAME"));
         }
      }
   }
}
```



B4.3 – Production Decision

For 12 campuses and 100 times the current number of passes, MySQL should be the primary system of record. A relational database supports concurrent counters, indexed searches, constraints, transactions, centralised permissions and backups. Text files are useful for import/export and audit snapshots but are weak as a shared transactional store. Java serialization is Java-specific, difficult to query and awkward to coordinate between concurrent writers.

Requirement	Text file	Serialized file	Relational database
Concurrent counters	Requires file locking	Requires locking/version coordination	Transactions and concurrency control
Pass ID search	O(n) unless indexed	Usually scan/deserialise or add index	Indexed lookup
Integrity	Application validation	Application validation/versioning	Keys, constraints and transactions
Multi-campus sharing	File sharing and coordination	File sharing and coordination	Central shared access
Evolution	Manual format migration	serialVersionUID and migration	Schema migration tools

The production design should use MySQL, PreparedStatement for all input, short transactions for single pass issuance and batch transactions for bulk imports. Index pass_id and route_no, use connection pooling, least-privilege accounts, backups and audit logging. If offline campus operation is required, queue local transactions with unique IDs and synchronise them with conflict handling after connectivity returns.

Alternative Concept – Separate Serialized File per Campus

Twelve independent campus files require opening and scanning or deserializing multiple files for a global search. With N total passes, aggregate search remains O(N), with extra file-open and deserialization overhead. Storage is O(N). Separate files reduce cross-campus write contention but each campus still needs locking or a single-writer queue for concurrent counters. Global duplicate detection and atomic cross-campus updates are difficult without a central transaction manager.

Final Evidence Summary

Evidence	Measured result	Decision supported
Buffering comparison	1 MB: 69.67× speed-up; 10 MB: 80.59×; 50 MB: 66.72×	Buffered I/O provides significantly faster file reading. Select buffered streaming I/O.
Search growth	1,000 records: 3.9354 ms; 10,000: 6.3406 ms; 50,000: 30.1887 ms; 100,000: 38.6329 ms	Sequential search time increases with dataset size. Indexing is beneficial for frequent searches on large datasets.
JDBC insert benchmark	10 records: Statement 53 ms, PreparedStatement 47 ms, Batch 42 ms. 1,000 records: 618 ms, 520 ms, 566 ms. 10,000 records: 2,440 ms, 2,253 ms, 2,385 ms. 100,000 records: 26,558 ms,	Batch insertion performed best for 100,000 records in this test.

	32,280 ms, 24,426 ms respectively.	
Database metadata	MySQL 9.4.0; MySQL Connector/J 26.10.0; tables app_user and bus_pass detected.	JDBC connectivity and database metadata inspection are successful.
Deployment compatibility	Successfully connected to the local MySQL database through Eclipse.	The application works in the tested local environment.

Conclusion

The assignment shows that persistence should be selected according to data shape, access pattern, reliability and scale. Buffered streams reduce small-read overhead while keeping memory bounded. DAO centralises SQL and object mapping. Serialization can preserve object graphs but requires care with transient resources, credentials and class evolution. JDBC transactions protect multi-step updates, while PreparedStatement prevents input from changing SQL structure.

The implementation tasks demonstrate that sequential file searches scale linearly, in-memory indexing trades memory for faster repeated lookup, and JDBC batching reduces round trips for bulk imports. For a multi-campus transport office with concurrent counters, a relational database is the strongest primary store, supported by validated files for import and export.